Global Solution 2026.2

AI Computer Systems & Sensors (AICSS)

Alerta Criadouro

Matheus Rebelo - RM 570260
Felipe Olmos - RM 570946
Luiz Guilherme - RM 573486

1. O problema e quem ele afeta

A dengue é um problema de saúde que começa dentro de casa. Segundo o governo do Estado de São Paulo, cerca de 75% dos focos do Aedes aegypti ficam dentro das residências: no pratinho do vaso, na calha entupida, no balde esquecido e na caixa d'água mal fechada (Poder360, fevereiro de 2026). Em 2024 o Brasil teve 6,48 milhões de casos prováveis e 5.972 mortes, o pior ano da história (Agência Brasil). Na cidade de São Paulo, a base do InfoDengue que usamos no projeto mostra 1.264.130 casos notificados só em 2024.

O que mais pesa é o tempo. A larva vira mosquito adulto em cerca de 10 dias. A orientação oficial é vistoriar a casa toda semana, mas na correria ninguém lembra, principalmente depois de um dia de chuva. Quase sempre a família só percebe que tinha um criadouro no quintal quando alguém já está doente.

O público que escolhemos são as famílias que moram em casas com quintal em bairros da cidade de São Paulo. Para elas, pensamos num aparelho simples que fica na área de serviço e avisa quando é hora de dar uma volta no quintal e esvaziar o que estiver com água, antes de o mosquito nascer.

2. Como a solução funciona

Ao lado da área de serviço, a céu aberto, fica um potinho sentinela. Ele pega chuva como qualquer vaso do quintal. Se o potinho juntou água, os outros recipientes do quintal provavelmente também juntaram. A boca do potinho tem uma tela fina, como de mosquiteiro: a chuva entra, mas o mosquito não consegue pôr ovos, então o próprio sensor não vira criadouro.

A caixa com o ESP32 fica na parede, sob o beiral, sem chuva e sem sol direto (o sol esquentaria o sensor e falsearia a temperatura). Um sensor ultrassônico fica preso numa haste, apontado para dentro do potinho.

A cada segundo o ESP32 lê a temperatura e a umidade do ar (DHT22) e a distância até a água do potinho (HC-SR04). Com isso ele calcula a altura da água, conta há quantos dias o potinho está com água e classifica a situação em OK, ATENÇÃO ou ALERTA. Depois acende o LED da cor certa, toca o buzzer quando precisa e manda um JSON com as leituras para a nossa API em Python, que mostra no log e grava num arquivo CSV.

3. Arquitetura e origem dos dados

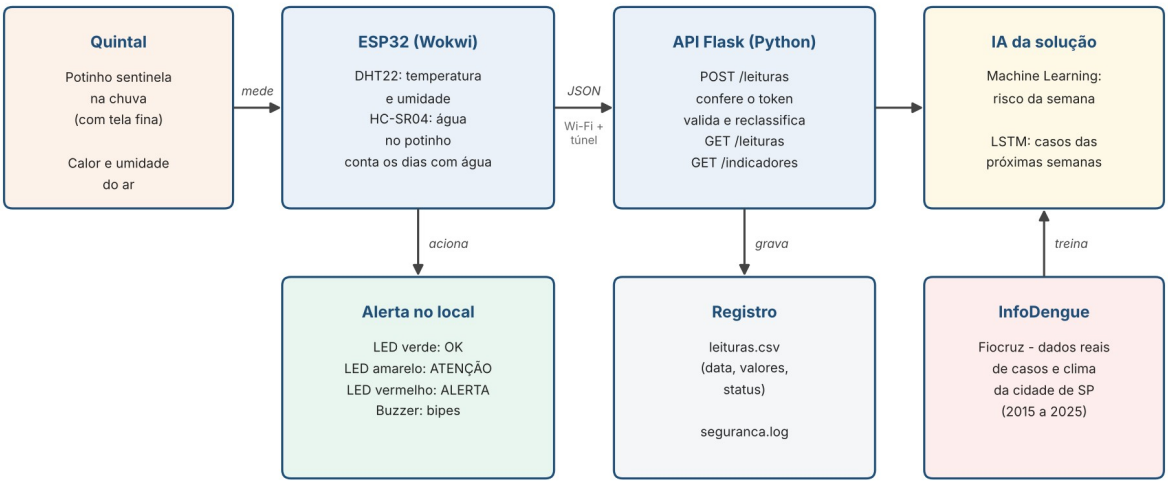


Figura 1 - Arquitetura do Alerta Criadouro

Os dados do dispositivo vêm dos sensores do ESP32, simulados no Wokwi. A API em Flask roda no Google Colab, e um túnel gratuito da Cloudflare dá um endereço público para o Wokwi conseguir mandar o JSON pela internet. O servidor grava cada leitura em leituras.csv. Os modelos de IA das outras matérias usam dados reais do InfoDengue (Fiocruz), com casos de dengue e clima da cidade de São Paulo semana a semana, de 2015 a 2025, para dizer o risco da semana na cidade.

4. O dispositivo no Wokwi

4.1 Componentes e ligações

Componente	Função	Pino do ESP32
DHT22	Temperatura (°C) e umidade do ar (%)	GPIO 15 (dados)
HC-SR04	Distância do sensor até a água do potinho (cm)	TRIG GPIO 5, ECHO GPIO 18
LED verde	Estado OK	GPIO 25 (com resistor de 220 Ω)
LED amarelo	Estado ATENÇÃO	GPIO 26 (com resistor de 220 Ω)
LED vermelho	Estado ALERTA	GPIO 27 (com resistor de 220 Ω)
Buzzer	Aviso sonoro	GPIO 13

4.2 Por que escolhemos esses sensores

O HC-SR04 foi escolhido porque o mosquito precisa de água parada para pôr ovos. O ultrassônico mede a distância até a superfície da água sem encostar nela, então não enferruja. Pela distância sabemos se o potinho tem água e quanta, e contando o tempo sabemos há quantos dias ela está parada, que é o que importa para o ciclo da larva.

O DHT22 entra porque o calor acelera o ciclo do mosquito. Entre 25 e 30 °C a larva vira mosquito mais rápido, e o ar úmido ajuda o mosquito adulto a viver mais. Preferimos o DHT22 ao DHT11 porque ele é mais preciso e mede numa faixa maior.

Os LEDs e o buzzer são o jeito de a família entender o aviso sem abrir aplicativo. Cor de semáforo todo mundo entende, inclusive crianças e idosos, e o buzzer chama a atenção de quem está dentro de casa.

Os dois sensores se completam: só a água parada não diz se o risco é alto, e só o clima não diz se existe criadouro. Juntos eles mostram quando o quintal está mesmo virando um berçário de mosquito.

4.3 Limiares e status de cada sensor

Cada sensor recebe um status OK ou ALERTA, que vai no JSON no campo status_sensores:

Sensor	OK	ALERTA	Por quê
Temperatura (DHT22)	abaixo de 25 °C	25 °C ou mais	Acima de 25 °C a larva se desenvolve mais rápido
Umidade (DHT22)	abaixo de 70%	70% ou mais	Ar úmido ajuda o mosquito a sobreviver e os ovos a não secarem
Água no potinho (HC-SR04)	menos de 1 cm	1 cm ou mais	Pouca água já basta para o mosquito pôr ovos

O potinho tem 12 cm do sensor até o fundo. A altura da água é calculada assim: nível da água = 12 - distância medida.

4.4 Decisão do estado geral

Estado	LED	Buzzer	Regra
OK	verde	desligado	Potinho seco
ATENÇÃO	amarelo	um bipe ao entrar	Tem água, mas ainda não chegou no ponto de alerta
ALERTA	vermelho	bipes repetidos	Água há 3 dias ou mais com calor e umidade altos, ou água há 5 dias ou mais em qualquer clima

Por que 3 e 5 dias: o ovo vira mosquito em cerca de 10 dias, e mais rápido no calor. Avisar no 3° dia (ou no 5°, quando está mais frio) dá tempo de a família esvaziar os recipientes antes de a larva virar mosquito.

No Wokwi não dá para esperar dias de verdade. Por isso, na simulação, 1 dia vale 10 segundos. No aparelho de verdade seria 1 dia de verdade, bastando mudar a constante MS_POR_DIA no código.

5. Transmissão dos dados em JSON

A cada 5 segundos, e sempre que o estado muda, o ESP32 monta e envia este JSON (exemplo real copiado do monitor serial do Wokwi, no momento do alerta):

```
{"dispositivo_id":"esp32-quintal-001","tipo_dispositivo":"esp32",
 "latitude":-23.541,"longitude":-46.471,
 "metrica":"dias_agua_parada","valor":3,
 "leituras":{"temperatura_c":29,"umidade_pct":78,"distancia_cm":7.1,"nivel_agua_cm":4.9},
 "status_sensores":{"temperatura":"ALERTA","umidade":"ALERTA","agua":"ALERTA"},
 "status_local":"ALERTA"}
```

O envio é um POST para a rota /leituras da API, com o cabeçalho X-Token-Dispositivo, que identifica o aparelho. A latitude e a longitude vão com só 3 casas decimais: isso mostra o bairro, não a casa exata da família. O formato do JSON é o mesmo usado na API de Python e na Lambda da AWS, então o sistema é um só.

A API devolve a resposta com o id da leitura gravada e a classificação que ela mesma calculou:

```
API respondeu HTTP 201
{"alerta":true,"dispositivo_id":"esp32-quintal-001","id":17,"metrica":"dias_agua_parada",
 "nivel_risco":"critico","status":"ALERTA","valor":3}
```

O servidor Python mostra cada leitura que chega no log (linha ACEITO, com dispositivo, dias, temperatura, umidade e status) e grava no CSV leituras.csv as colunas data_hora, dispositivo_id, temperatura_c, umidade_pct, distancia_cm, nivel_agua_cm, dias_agua_parada, status, nivel_risco e alerta.

6. Evidências: os três comportamentos

Rodamos a simulação no Wokwi no dia 05/11/2026 com o DHT22 em 29 °C e 78% de umidade (um dia típico de verão em São Paulo) e fomos mudando a distância do ultrassônico para simular o potinho enchendo de água.

Comportamento 1 - Potinho seco: OK

Distância de 12 cm (potinho vazio). O LED verde acende e o buzzer fica quieto. Mesmo com o clima quente e úmido (temperatura e umidade em ALERTA), o estado geral é OK, porque não há água para o mosquito.

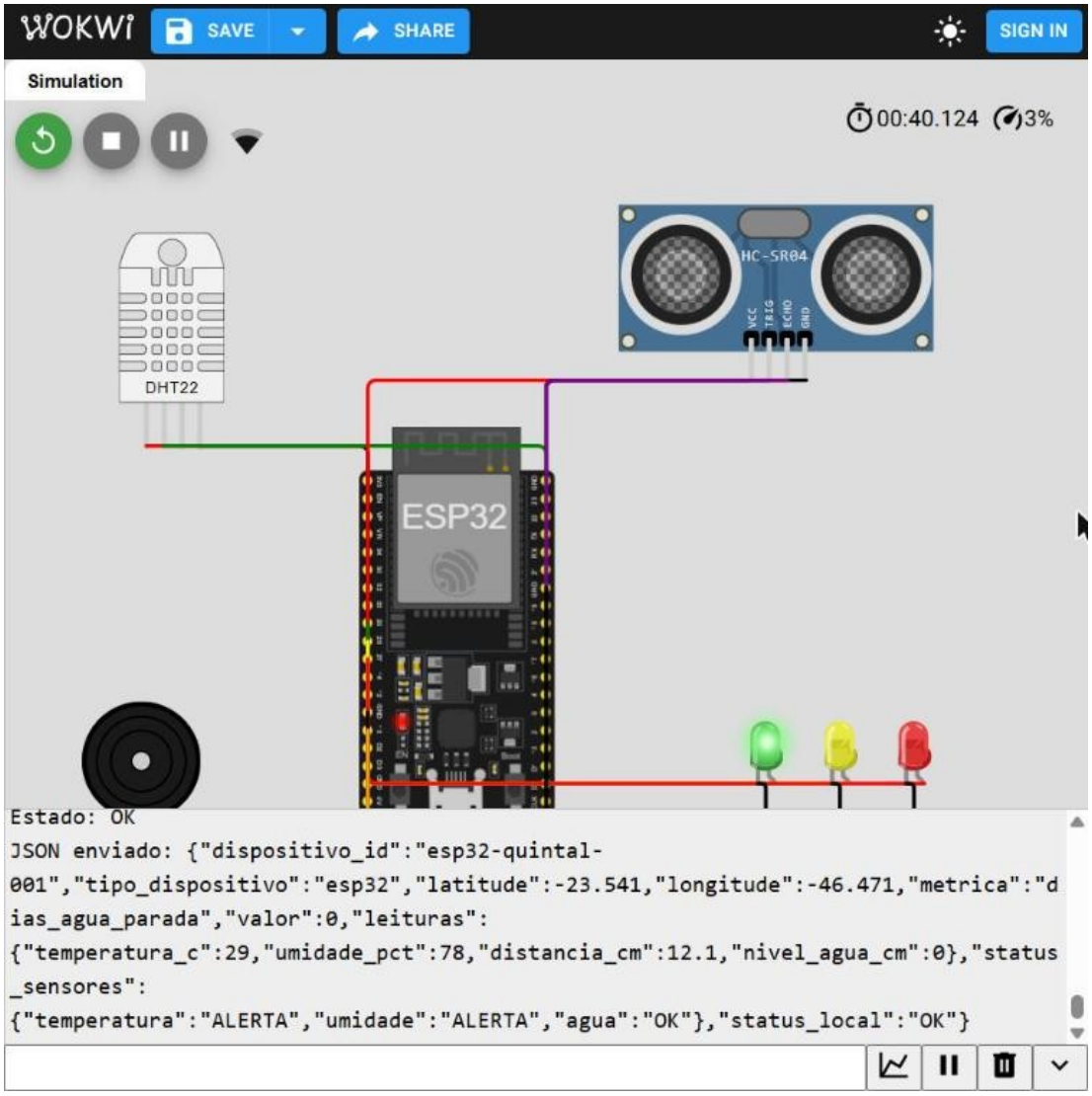


Figura 2 - LED verde aceso e JSON com valor 0 e status_local OK

Comportamento 2 - Água parada: ATENÇÃO

Mudamos a distância para 7 cm, como se tivesse chovido e o potinho tivesse 4,9 cm de água. O sensor de água vai para ALERTA, o LED amarelo acende e o buzzer dá um bipe. O JSON sai com valor 1 (primeiro dia com água) e status_local ATENCAO.

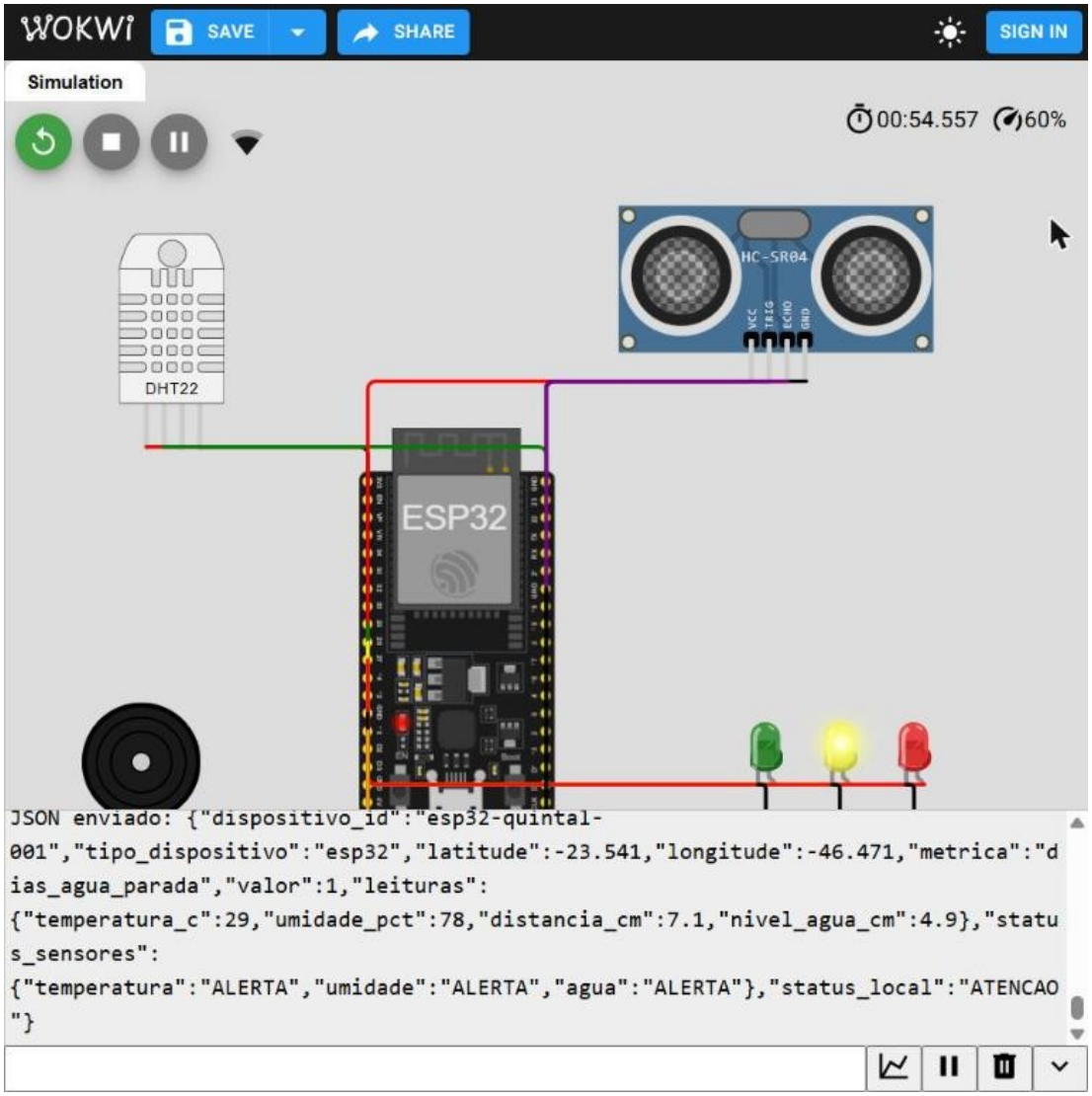


Figura 3 - LED amarelo aceso e JSON com status_local ATENCAO

Comportamento 3 - Água há 3 dias com calor: ALERTA

Deixamos a água no potinho. No 3° dia (30 segundos de simulação), com calor e umidade altos, o estado vira ALERTA: LED vermelho aceso e buzzer com bipes repetidos. A API respondeu HTTP 201 com nivel_risco critico e alerta true.

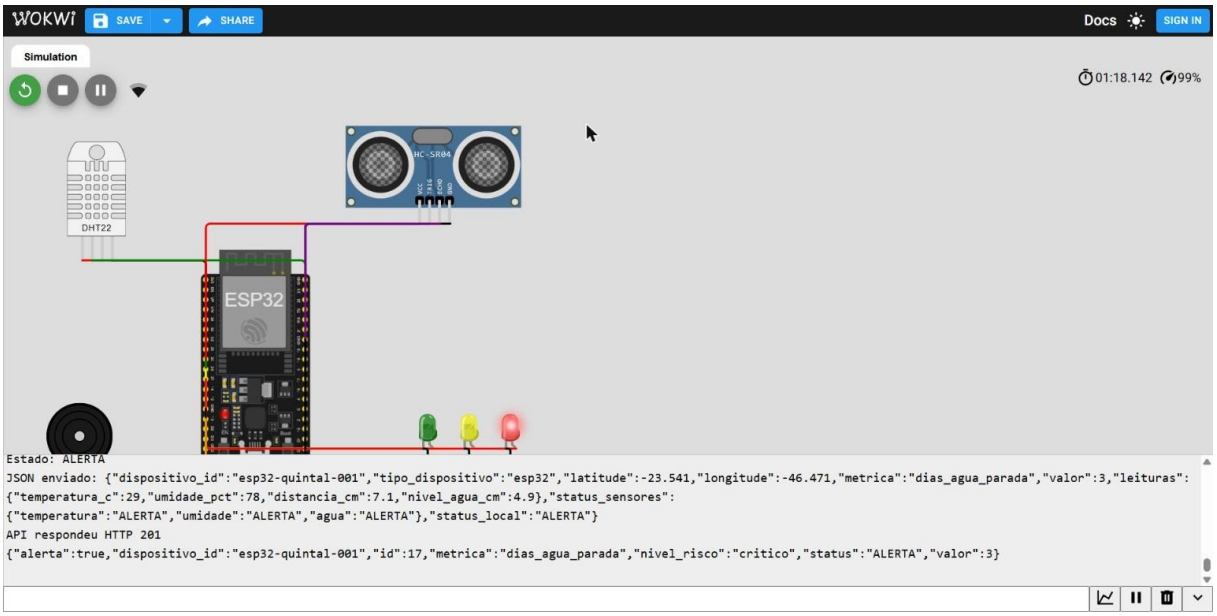


Figura 4 - LED vermelho aceso, JSON do alerta e resposta da API

Registro no servidor

No Colab, o arquivo leituras.csv mostra a sequência que veio do Wokwi: as leituras OK, depois ATENCAO nos dias 1 e 2 e ALERTA a partir do dia 3. As três primeiras linhas (18:26:07) são os testes automáticos da API feitos antes de ligar o Wokwi.

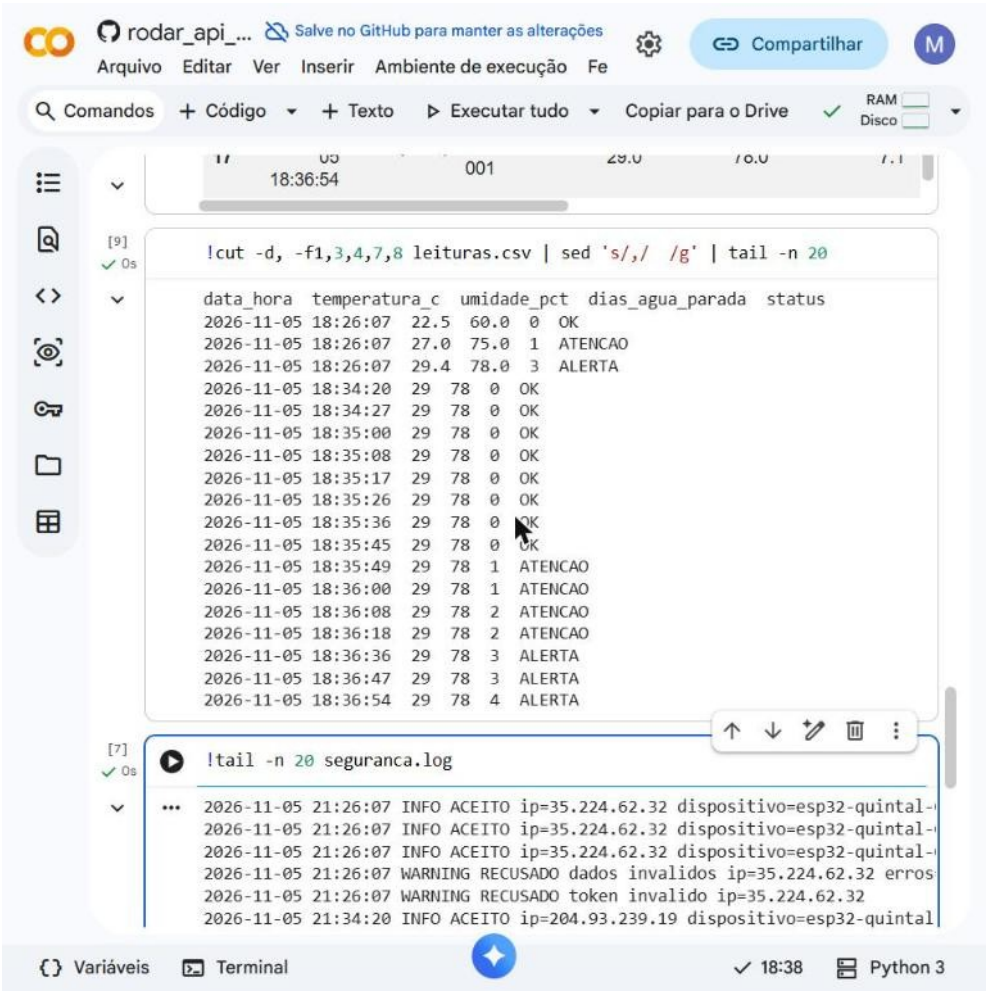


Figura 5 - Leituras gravadas no CSV pelo servidor Python

No print, o horário do arquivo seguranca.log aparece em UTC (21:26), 3 horas a mais que o de São Paulo. O CSV já grava no horário de São Paulo (18:26). Depois desse teste ajustamos o código para o log também usar o horário de São Paulo.

Entrada (Wokwi)	LED / buzzer	JSON (valor e status)	Registro no CSV
29 °C, 78%, distância 12 cm	verde / quieto	valor 0, OK	18:34:20 a 18:35:45 - OK
29 °C, 78%, distância 7 cm (dias 1 e 2)	amarelo / um bipe	valor 1 e 2, ATENCAO	18:35:49 a 18:36:18 - ATENCAO
29 °C, 78%, distância 7 cm (dia 3 em diante)	vermelho / bipes	valor 3 e 4, ALERTA	18:36:36 a 18:36:54 - ALERTA

7. O que foi simulado e as limitações

Algumas coisas foram simuladas e outras foram implementadas de verdade. Também encontramos limitações que precisam ser ditas:

- Simulado: o ESP32, os sensores, os LEDs e o buzzer rodam no Wokwi. A chuva enchendo o potinho foi simulada mudando a distância do ultrassônico. Os dias foram acelerados (1 dia = 10 segundos).
- Implementado de verdade: o código do ESP32, o envio do JSON pela internet, a API em Flask com validação, token e limite de requisições, e a gravação no CSV.

- O aparelho não enxerga cada vaso da casa. Ele funciona como um lembrete inteligente: o potinho representa os outros recipientes do quintal, mas uma caixa d'água destampada, por exemplo, pode ter água mesmo sem chuva.
- Conexão: no Wokwi o envio foi por http, porque a conexão segura (https) estourava o tempo dentro do simulador. No aparelho real o envio seria por https. O endereço do túnel muda toda vez que o notebook do Colab é ligado.
- Sensor ao ar livre: na vida real, sujeira, folhas ou insetos dentro do potinho podem atrapalhar a leitura do ultrassônico. Por isso a tela fina e uma limpeza de vez em quando são importantes.
- Energia e Wi-Fi: o aparelho precisa de tomada e de Wi-Fi. Sem Wi-Fi, o alerta local (LED e buzzer) continua funcionando, só não envia os dados.

8. Inovação

Hoje, combater a dengue em casa depende de alguém lembrar de olhar o quintal ou da visita do agente de saúde. No Alerta Criadouro é o próprio quintal que avisa quando é hora de olhar. A ideia do potinho sentinela é nossa forma de medir um quintal inteiro com um sensor só, de um jeito barato, e a tela fina garante que o aparelho nunca vira um criadouro. Além disso, o aviso considera ao mesmo tempo a água parada, há quantos dias ela está lá e o clima, em vez de só dizer se tem água ou não. Os dados de várias casas, juntos, podem mostrar à prefeitura e aos agentes de combate a endemias quais bairros estão com mais água parada depois da chuva.

9. Relação com os ODS

- ODS 3 - Saúde e bem-estar: menos criadouros em casa significa menos casos de dengue e menos mortes.
- ODS 11 - Cidades e comunidades sustentáveis: a prevenção acontece no bairro, junto com os agentes de saúde e a prefeitura.
- ODS 13 - Ação contra a mudança do clima: calor e chuva fortes aumentam a dengue. O aparelho usa esses dados do clima para avisar no momento certo.
- ODS 6 - Água potável e saneamento (complemento): o projeto ensina o cuidado com a água parada em casa.

10. Código e como reproduzir

Todo o código está no GitHub: https://github.com/matheusrebelo04-maker/alerta-criadouro

- dispositivo/sketch.ino: código do ESP32 (no Apêndice A).
- dispositivo/diagram.json: circuito do Wokwi.
- dispositivo/libraries.txt: bibliotecas: DHT sensor library for ESPx e ArduinoJson.
- api/app.py e api/regras.py: servidor Python que recebe, valida, mostra e grava as leituras.
- api/rodar_api_colab.ipynb: notebook que liga a API no Colab e abre o túnel.

Passo a passo: (1) abrir o notebook api/rodar_api_colab.ipynb no Google Colab e rodar todas as células; (2) copiar o endereço que aparece no fim da célula 2; (3) criar um projeto ESP32 no wokwi.com, colar o sketch.ino e o diagram.json, adicionar as duas bibliotecas e trocar o URL_API pelo endereço copiado (com http); (4) iniciar a simulação e mudar a distância do HC-SR04 clicando nele.

Referências

- Poder360. 75% dos focos do mosquito da dengue estão dentro das casas, diz governo de SP. Fevereiro de 2026.
- Agência Brasil. Brasil registra 6,48 milhões de casos prováveis de dengue em 2024, o pior ano da história.
- Metrópoles. Ciclo do Aedes aegypti: do ovo ao mosquito adulto em cerca de 10 dias.
- InfoDengue (Fiocruz e FGV). Dados semanais de dengue e clima da cidade de São Paulo (geocode 3550308). https://info.dengue.mat.br
- Wokwi. Simulador de ESP32. https://wokwi.com

Apêndice A - Código do ESP32 (sketch.ino)

```
// Alerta Criadouro - dispositivo do quintal (ESP32)
// Global Solution 2026.2 - FIAP - 1TIAPZ
// Matheus Rebelo - RM 570260
// Felipe Olmos - RM 570946
// Luiz Guilherme - RM 573486
//
// DHT22 mede temperatura e umidade. O HC-SR04 aponta para dentro do
// potinho sentinela e mede a distância até a água. LEDs e buzzer avisam
// a família. A cada leitura o ESP32 manda um JSON para a nossa API.

#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>
#include "DHTesp.h"

// ---------- rede e API ----------
const char* WIFI_NOME = "Wokwi-GUEST";
const char* WIFI_SENHA = "";
// endereço da API: túnel aberto pelo notebook api/rodar_api_colab.ipynb
// (a cada vez que o notebook roda, o Colab gera um endereço novo; trocar aqui)
// No Wokwi usamos http porque a conexão segura estoura o tempo no simulador.
// No aparelho de verdade o envio seria por https.
const char* URL_API = "http://vsnet-adequate-spaces-era.trycloudflare.com/leituras";
const char* TOKEN_DISPOSITIVO = "ac-quintal-001-demo";
const char* ID_DISPOSITIVO = "esp32-quintal-001";

// ---------- pinos ----------
const int PINO_DHT = 15;
const int PINO_TRIG = 5;
const int PINO_ECHO = 18;
const int LED_VERDE = 25;
const int LED_AMARELO = 26;
const int LED_VERMELHO = 27;
const int PINO_BUZZER = 13;

// ---------- limiares (iguais aos da API) ----------
const float ALTURA_POTINHO_CM = 12.0;  // sensor até o fundo vazio
const float AGUA_MINIMA_CM = 1.0;
const float TEMP_ALERTA = 25.0;
const float UMIDADE_ALERTA = 70.0;
const int DIAS_ALERTA_CLIMA = 3;
const int DIAS_ALERTA_SEMPRE = 5;

// no Wokwi 1 dia vale 10 segundos, para dar para ver acontecer
const unsigned long MS_POR_DIA = 10000;
const unsigned long INTERVALO_ENVIO = 5000;

DHTesp dht;
unsigned long inicioAgua = 0;   // quando o potinho começou a ter água
bool temAgua = false;
unsigned long ultimoEnvio = 0;
String estadoAnterior = "";

float medirDistancia() {
  digitalWrite(PINO_TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(PINO_TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(PINO_TRIG, LOW);
  long duracao = pulseIn(PINO_ECHO, HIGH, 30000);
  if (duracao == 0) return -1;
  return duracao * 0.0343 / 2.0;
}

String classificar(int dias, float temp, float umid) {
  if (dias <= 0) return "OK";
  if (dias >= DIAS_ALERTA_SEMPRE) return "ALERTA";
  bool climaFavoravel = temp >= TEMP_ALERTA && umid >= UMIDADE_ALERTA;
  if (dias >= DIAS_ALERTA_CLIMA && climaFavoravel) return "ALERTA";
  return "ATENCAO";
}

void mostrarEstado(String estado) {
  digitalWrite(LED_VERDE, estado == "OK");
  digitalWrite(LED_AMARELO, estado == "ATENCAO");
  digitalWrite(LED_VERMELHO, estado == "ALERTA");

  if (estado == "ALERTA") {
```

```
    // bipes repetidos enquanto estiver em alerta
    for (int i = 0; i < 3; i++) {
      tone(PINO_BUZZER, 2000, 150);
      delay(250);
    }
  } else if (estado == "ATENCAO" && estadoAnterior != "ATENCAO") {
    tone(PINO_BUZZER, 1000, 200);  // um bipe só quando entra em atenção
  }
}

void enviarJson(String corpo) {
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("Sem Wi-Fi, leitura nao enviada");
    return;
  }
  WiFiClient cliente;
  HTTPClient http;
  http.setConnectTimeout(15000);  // o simulador às vezes demora para conectar
  http.setTimeout(15000);
  http.begin(cliente, URL_API);
  http.addHeader("Content-Type", "application/json");
  http.addHeader("X-Token-Dispositivo", TOKEN_DISPOSITIVO);
  int codigo = http.POST(corpo);
  Serial.print("API respondeu HTTP ");
  Serial.println(codigo);
  if (codigo > 0) Serial.println(http.getString());
  http.end();
}

void setup() {
  Serial.begin(115200);
  dht.setup(PINO_DHT, DHTesp::DHT22);
  pinMode(PINO_TRIG, OUTPUT);
  pinMode(PINO_ECHO, INPUT);
  pinMode(LED_VERDE, OUTPUT);
  pinMode(LED_AMARELO, OUTPUT);
  pinMode(LED_VERMELHO, OUTPUT);
  pinMode(PINO_BUZZER, OUTPUT);

  Serial.print("Conectando no Wi-Fi");
  WiFi.begin(WIFI_NOME, WIFI_SENHA);
  int tentativas = 0;
  while (WiFi.status() != WL_CONNECTED && tentativas < 40) {
    delay(250);
    Serial.print(".");
    tentativas++;
  }
  Serial.println(WiFi.status() == WL_CONNECTED ? " conectado!" : " sem Wi-Fi, segue so no alerta
local");
}

void loop() {
  TempAndHumidity clima = dht.getTempAndHumidity();
  float distancia = medirDistancia();
  if (isnan(clima.temperature) || distancia < 0) {
    Serial.println("Leitura invalida, tentando de novo");
    delay(1000);
    return;
  }

  float nivel = ALTURA_POTINHO_CM - distancia;
  if (nivel < 0) nivel = 0;

  // conta há quantos dias tem água no potinho
  if (nivel >= AGUA_MINIMA_CM) {
    if (!temAgua) {
      temAgua = true;
      inicioAgua = millis();
    }
  } else {
    temAgua = false;
  }
  int dias = temAgua ? (millis() - inicioAgua) / MS_POR_DIA + 1 : 0;

  String estado = classificar(dias, clima.temperature, clima.humidity);
  mostrarEstado(estado);

  if (millis() - ultimoEnvio >= INTERVALO_ENVIO || estado != estadoAnterior) {
    JsonDocument doc;
    doc["dispositivo_id"] = ID_DISPOSITIVO;
    doc["tipo_dispositivo"] = "esp32";
```

```
    doc["latitude"] = -23.541;
    doc["longitude"] = -46.471;
    doc["metrica"] = "dias_agua_parada";
    doc["valor"] = dias;
    JsonObject leituras = doc["leituras"].to<JsonObject>();
    leituras["temperatura_c"] = round(clima.temperature * 10) / 10.0;
    leituras["umidade_pct"] = round(clima.humidity * 10) / 10.0;
    leituras["distancia_cm"] = round(distancia * 10) / 10.0;
    leituras["nivel_agua_cm"] = round(nivel * 10) / 10.0;
    JsonObject sensores = doc["status_sensores"].to<JsonObject>();
    sensores["temperatura"] = clima.temperature >= TEMP_ALERTA ? "ALERTA" : "OK";
    sensores["umidade"] = clima.humidity >= UMIDADE_ALERTA ? "ALERTA" : "OK";
    sensores["agua"] = nivel >= AGUA_MINIMA_CM ? "ALERTA" : "OK";
    doc["status_local"] = estado;

    String corpo;
    serializeJson(doc, corpo);
    Serial.println("----------------------------------------");
    Serial.print("Estado: ");
    Serial.println(estado);
    Serial.print("JSON enviado: ");
    Serial.println(corpo);
    enviarJson(corpo);
    ultimoEnvio = millis();
  }

  estadoAnterior = estado;
  delay(1000);
}
```